

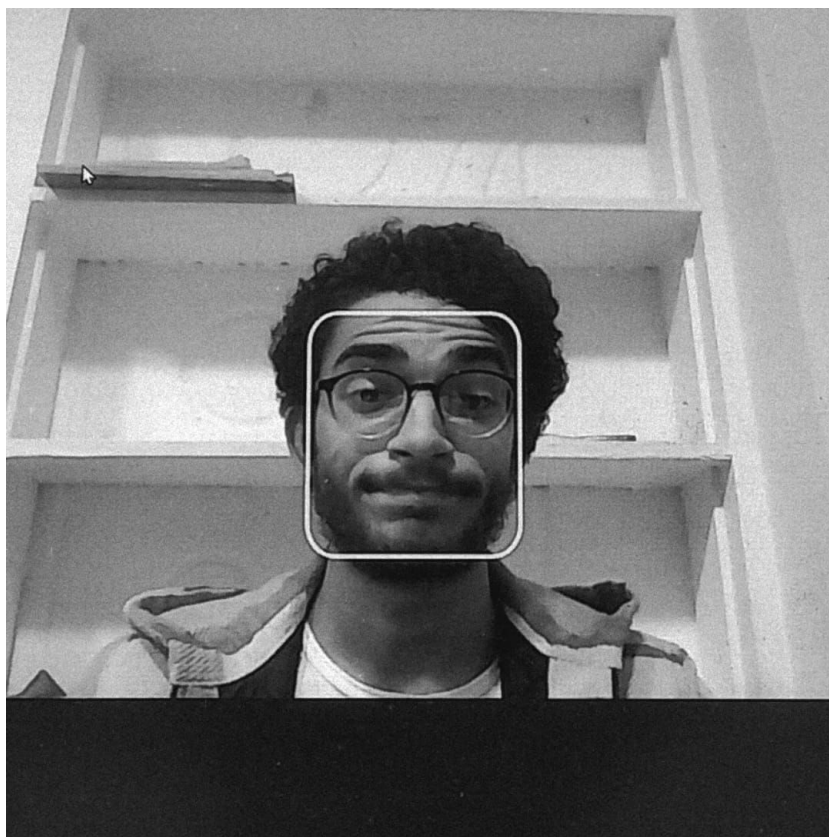
Backend Engineering

Workshop • Session 1 of 2

Today's journey

What backend engineering is • What backend engineers actually do • Engineer vs. developer • Foundations • The backend stack

who am I ?



What is Backend Engineering?

The systems behind the software you use

Definition

Backend engineering is the design, construction, and operation of the server-side systems that power software.

Frontend

What the user touches: buttons, screens, forms.

Backend

What makes those actions mean something: data, permissions, processing, and business logic.

A Beginner-Friendly Mental Model

Think of an application like a restaurant

Frontend = Dining Room

The part customers interact with.

Backend = Kitchen

Where requests are processed and decisions happen.

Backend also includes

Storage, suppliers, operations, security, and the systems keeping everything running.

The important idea: backend work is often invisible by design.

You see the result; the backend handles the work behind it.

Backend Vocabulary

Terms you will hear throughout the workshop

API

A contract that lets other software talk to your system

VPS

A rented virtual server you control

protocols

Rules for communication between systems.

DNS

Translates domain names into IP addresses

Request / Response

Client sends a request → server processes it → server responds

Domain

A human-readable name that maps to a server address

Hosting

Where the application runs and becomes reachable

URL

The full address used to locate a web resource

Task

**Think of one app you used today.
What is ONE thing happening on its backend right now that you cannot see?**

What Does a Backend Engineer Actually Do?

The job is much bigger than writing endpoints

APIs

Build contracts other teams can depend on. Version APIs without breaking users.

Data

Design relationships, queries, and safe changes as data grows.

Auth & Security

Know who the user is and what they are allowed to do.

Background Work

Move slow work to queues/workers; handle retries and duplicate jobs.

Integrations

Connect reliably to payments, messaging, maps, identity, and other systems.

Reliability

Use logs, metrics, tracing, alerts, caching, and connection pools.

Infrastructure

Deploy outside the laptop, keep systems running, and understand cost.

Authentication ≠ Authorization

Authentication

WHO are you?

Authorization

WHAT are you allowed to do?

Authentication can involve sessions, tokens, password storage, or API keys.

Authorization can involve roles and permissions.

Security problems often appear when these concepts are confused.

Background Work & Idempotency

Why background work?

Some operations are too slow to make a user wait: emails, uploads, payments, reports.

The problem

A job may fail, retry, or accidentally run twice.

Key word: Idempotency

Design a job so repeating it does not create an incorrect result.

Responsibility Scenarios

Map real problems to backend responsibility buckets

Payment provider down

Integrations + Reliability

Private messages exposed

Authentication / Authorization + Security

45-second report times out

Background Work + Performance

Backend Engineer vs. Backend Developer

The titles often overlap — ownership is the useful distinction

Backend Developer

Implements a specification: endpoint, query, test. The work is usually bounded and the system shape is often decided by someone else.

Backend Engineer

Owens decisions about system shape, scale, failures, migrations, dependencies, and operational responsibility.

* Learning to develop can get you a first job, Learning to engineer means learning to own the consequences of your decisions.

Task

Think of a project you built.

Write one decision someone else specified for you, and one decision YOU made because you owned the problem.

Foundational Knowledge

The core subjects behind backend engineering

Programming Fundamentals

Write correct, maintainable programs

Data Structures

Choose efficient ways to store and access data

Databases

Store, relate, query, and evolve data

Networking

Understand how requests travel and why they fail

OOP

Model and organize software behavior

Algorithms

Choose effective ways to solve problems

Operating Systems

Understand processes, resources, files, and limits

Nice to have

Design patterns and SOLID — useful later, not urgent now

Why Do the Foundations Matter?

Each one prevents a class of real problems

Data Structures

Wrong structure → a lookup that should be fast can become $O(n)$.

Operating Systems

Without OS knowledge, resource limits such as file descriptors can be mysterious.

Networking

Without networking knowledge, it is hard to explain why a request hangs versus fails.

The goal is not memorizing theory.

The goal is understanding what is happening when the system behaves unexpectedly.

Task

Show of hands: which foundational topics have you NEVER touched?
Programming • OOP • Data Structures • Algorithms • Databases • OS •
Networking

The Backend Engineer Stack

Five categories — many possible choices

1

Language

js / Java / PHP / Python / ...

2

Framework

.NET / NestJS / Spring / Laravel / ...

3

Database

PostgreSQL / MySQL / MongoDB / ...

4

Cache & Queue

Redis / message queues / ...

5

Infrastructure

VPS / cloud / hosting / deployment

Example Stack

Same five decisions, made differently by every company



The categories stay recognizable even when the technologies change.

Q / A

my question

What part of backend engineering is still unclear?